Una plaza, una venta válida

Base de datos de aerolínea · PAC PSMA00611

- Participantes activos y líder: [POR COMPLETAR]
- SQL Server + Python; datos sintéticos en USD
- Versión preparada: ejecución SQL y Azure pendientes

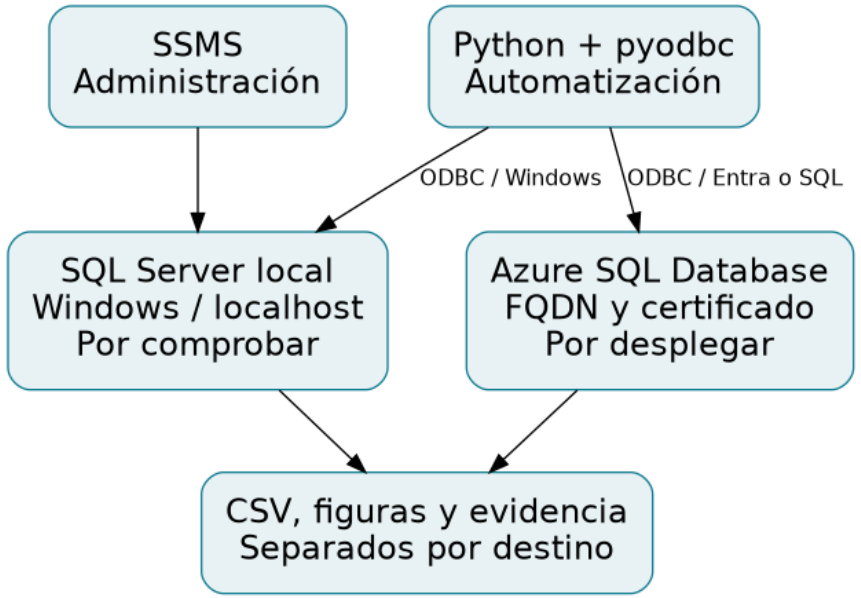
Problema, objetivos y alcance

- Comprar sin registro; comprador y pasajero pueden diferir
- Evitar dos reservas activas del mismo asiento
- Distinguir contratos, cobros y reembolsos
- Alcance: un pasajero y un vuelo por tiquete; dos cabinas

Requisitos con fuente y límites

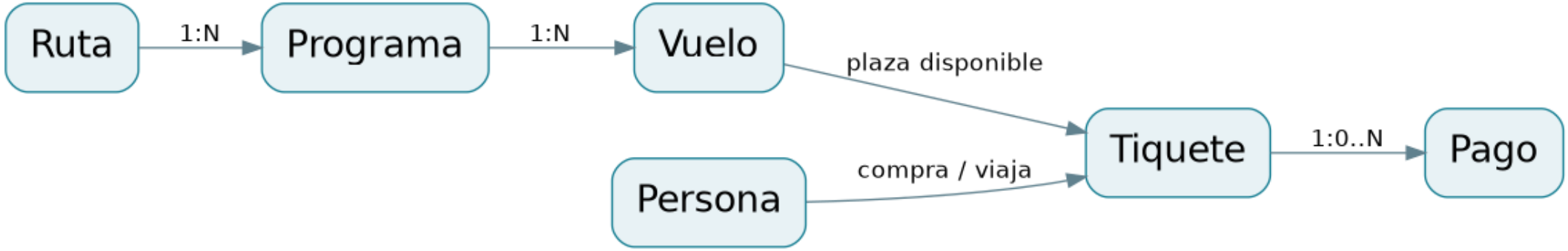
- Nube, modelo, tablas, DML, vistas y procedimientos
- GitHub; 10–15 diapositivas, 20 minutos, exposición aleatoria
- 46 filas de trazabilidad; fuentes separadas de supuestos
- Calendario discrepante y mínimo de páginas sin cifra

Arquitectura reproducible



- SSMS administra; SQL Server almacena; Python automatiza
- Windows local y Azure son destinos con evidencia separada

Del concepto a las relaciones



- Persona → tiquete → pagos → reembolso
- Ruta → programación → vuelo → plaza

Normalización e historia comercial

- Dependencias funcionales y claves candidatas explícitas
- Sin listas de pagos ni repetición de aeropuertos por venta
- Precio de venta es un hecho histórico
- Excepciones físicas: AircraftId, CabinCode y precio calculado

Integridad de plazas y recursos

- Inventario por asiento; capacidad por cabina = conteo
- FK + CHECK + índice único de asiento activo
- Transacción + bloqueo de escritura compartido por la API
- Avión y tripulación: intervalos sin solapar, más 45 minutos

Venta, cobro y cancelación atómicos

- HELD → PARTIAL → PAID; cancelación conserva historia
- Pago fallido no cuenta; cobro confirmado no excede saldo
- UUID estable evita duplicar reintentos
- Cancelación devuelve cobros y libera plaza en una transacción

Python y datos reproducibles

- Semilla 6112026; 32 vuelos y 896 solicitudes planeadas
- pyodbc + parámetros; GO procesado por el cliente
- CSV y figuras solo a partir de consultas ejecutadas
- Hashes y destino en cada manifiesto de resultados

Análisis que concilia

- 18 consultas: ocupación, canales, saldos, caja y flota
- Pagos agregados por tiquete antes del JOIN
- Ventas netas = caja neta + saldo, según la cohorte
- Sin costos no se calcula rentabilidad

Pruebas: evidencia y pendientes

- 21 pruebas offline superadas; 18 SELECT analizados parcialmente
- 34 pruebas SQL preparadas y pendientes
- Dos sesiones competirán por la última plaza
- Motor remoto: descarga bloqueada; laptop aún no comprobada

Azure y demostración verificable

- Desarrollo local primero; despliegue Azure aún pendiente
- Certificado validado y acceso limitado al cliente
- Evidencia necesaria: EngineEdition=5 + consultas y pruebas
- GitHub se publica solo con autorización del equipo

Conclusiones y cierre del equipo

- Modelo e implementación integral, con decisiones justificadas
- Reproducibilidad offline comprobada; SQL por verificar
- Completar pruebas, evidencia Azure y aportes reales
- Todos deben defender modelo, consulta, regla y limitación